Measurement-Driven Routing of Text and Multimodal Requests Across Heterogeneous CPU, Laptop, and GPU Endpoints

CSE 291A Fall 2026 • Option 3: Adaptive Routing Across Heterogeneous Machines • Phase 1 Report

[Team member names]

1 Problem and Research Question

Inference fleets are increasingly heterogeneous: cheap cloud CPUs, laptops with integrated accelerators, and commodity GPUs differ in compute, memory bandwidth, capacity, and distance from the client. These differences change request completion time, but they do not by themselves justify a complex router. We serve one model family, Qwen3.5 [2], which is natively image–text-to-text and spans dense 0.8B/2B/4B/9B/27B and MoE 35B-A3B variants, on such a fleet and ask:

Core question. *When should a text or image request run on each machine, given its shape (modality, prompt and image size, output length, model tier), current queueing, and the cost of moving the request and its context over the network?*

Phase 1 establishes the differences experimentally; Phase 2 derives a router from them. We test four hypotheses:

H1 *Shape dependence.* The unloaded endpoint ranking depends on request shape, because prefill (compute-bound), decode (memory-bandwidth-bound), and vision encoding scale differently across CPU ISAs, core counts, and accelerators.

H2 *Communication regimes.* Transfer cost is negligible for text inside one AWS Region but not for image payloads sent over the WAN to off-cloud endpoints.

H3 *Contention.* Under load the fastest endpoint saturates, slower endpoints become useful, and speed-oblivious policies (round-robin, join-shortest-queue) misroute [3].

H4 *Session state.* Qwen3.5 interleaves three Gated DeltaNet (linear-attention, fixed-size state) layers per full-attention layer, so per-session state is small; this shifts when migrating a multi-turn session beats keeping affinity.

2 Environment and Endpoints

Table 1: Endpoints. A–C and the router share one AWS Region/AZ; laptops and the GPU host are off-cloud. Instance types pending eligibility check.

ID	Hardware	Models	Isolates
A	`c7g.xlarge`: Graviton3, 4 cores, 8 GB	0.8B–4B	–
B	`c7g.2xlarge`: Graviton3, 8 cores, 16 GB	0.8B–4B	A vs B: cores/mem
C	`c7i.2xlarge`: Xeon SPR, 4C/8T, 16 GB, AMX	0.8B–4B	B vs C: ISA
L	Team laptops (up to 5): **[chip/RAM]**	0.8B–9B	edge tier
G	Commercial GPU host: **[GPU/VRAM]**	0.8B–35B-A3B	accel. tier
R	`c7g.medium`: router + load generator	–	–

Core and extended study. The core study follows the option specification: at least two AWS CPU instance types plus the router in one AWS Region, communicating over private IPs (Table 1). A–C are chosen so that each pair changes one factor. A vs. B varies core count and memory on the same ISA. B vs. C varies the ISA (Arm Neoverse-V1 vs. x86 with AVX-512/AMX) at equal vCPU count and memory; because an x86 vCPU is a hyperthread, C has half of B's physical cores. The extended study adds team laptops and a commercial GPU host for the larger tiers. The router reaches them over the WAN through a WireGuard (Tailscale) tunnel, which deliberately creates a second communication regime.

Runtime. Every endpoint runs the same pinned llama.cpp release [1] (`llama-server` with an OpenAI-compatible API, and images through the model's `mmproj` projector). It is built natively per backend (Arm NEON/SVE, x86 AVX-512/AMX, Metal, CUDA) and loads identical GGUF files verified by SHA-256. Thinking mode stays disabled, which is Qwen3.5's default.

Confounder control. The 0.8B and 2B *anchor* models run on every endpoint at the same quantization (Q4_K_M), which gives matched-model hardware comparisons. Each endpoint's best practical configuration (threads, quant format, backend) is reported separately. Larger tiers (9B–35B-A3B) run only off-cloud and are used in the optional model-tier extension under an explicit quality objective, so gains from a smaller model are never credited to routing.

3 Workloads

Request classes. Text: **T1** short chat; **T2** prefill-heavy (2–8K-token document, short answer); **T3** decode-heavy (short prompt, long output); **T4** agent sessions (k dependent steps with accumulating context and sampled tool delays). Multimodal: **M1** single image plus a short question at three resolutions (the visual-token count grows with resolution); **M2** chart or document image with a longer answer; **M3** multi-image prompts.

Sources. Length distributions come from public chat traces (ShareGPT-style). Quality probes use programmatic answer checks: GSM8K and MMLU subsets for text, and ChartQA and TextVQA subsets for images.

Arrivals and splits. Arrivals are open-loop Poisson (steady) and on/off bursty with the same mean rate, generated from fixed seeds into replayable traces. Open-loop arrivals avoid hiding queueing delay. Policies are designed and tuned on *dev* mixes only. Evaluation uses *held-out* mixes with different text:image ratios and length distributions.

Output control. Performance runs fix output length (`n_predict` with `ignore_eos`) so that every endpoint generates identical token counts; quality runs terminate naturally.

4 Metrics and Instrumentation

Request level. The router and the load generator log one JSONL record per request. Arrival, routing, first-token, and

completion timestamps give TTFT, TPOT (mean inter-token time after the first token), end-to-end latency, and router queueing delay. Each record also holds the server-reported prompt/image/output token counts, prefill and decode times, payload bytes, and status (ok/timeout/error). Failures are recorded, never dropped.

Endpoint level. A 1 Hz sampler records CPU utilization, memory (RSS), GPU utilization and memory (`nvidia-smi`; `macmon` on Apple Silicon), and `llama-server` slot occupancy.

Aggregates. Throughput (req/s and output tok/s); p50/p95/p99 TTFT and TPOT; SLO attainment (goodput); utilization balance across endpoints; network bytes; AWS cost per 1K requests.

Hygiene. Model, runtime, and AMI versions are pinned. Each experiment runs at least three times, and we report medians with bootstrap 95% CIs. Cold start (model and projector load, first request) is reported separately from warm steady state. Laptops run on AC power in a fixed performance mode, with background load minimized and a cool-down between runs.

5 Phase 1 Experiments

E1 *Isolated profiles:* endpoint × model × quant × prompt (128–8K) × output (32–1K) × image resolution → prefill rate, TPOT vs. context depth, vision-encode time, and peak memory.

E2 *Load curves:* per endpoint, sweep slots (1–8) and Poisson rate up to saturation → TTFT/TPOT vs. load, capacity knee, and utilization.

E3 *Communication:* RTT and bandwidth (intra-AZ vs. WAN), transfer time vs. payload (text context, image bytes), and router overhead (direct vs. routed at equal load).

E4 *Session state:* per-turn latency with warm affinity vs. a forced endpoint switch, and whether prefix reuse works for Qwen3.5's hybrid layers in the pinned runtime.

E1–E4 yield a fitted per-endpoint service-time model and crossover maps (the best endpoint per request class, idle vs. loaded). Together they answer whether one endpoint dominates when unloaded and whether another becomes useful under contention. Table 2 reports initial results.

Table 2: Initial measurements: Qwen3.5-2B Q4_K_M, warm, one request, 512-token prompt (image: M1 mid-res.), median of [*n*] runs.

	Prefill (tok/s)	TPOT (ms)	TTFT text (ms)	TTFT image (ms)	Peak mem (GB)	RTT (ms)
A	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
B	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
C	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
L	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
G	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

6 Phase 2 Direction: Completion-Time Routing

We plan an estimated-completion-time (ECT) router. For request r and each feasible endpoint i:

$$\mathrm{ECT}_i(r) = W_i + E_i(v_r) + P_i(p_r) + \hat{o}_r\,\mathrm{TPOT}_i(b_i) + \mathrm{RTT}_i + s_r/\mathrm{BW}_i + C_i(r). \quad (1)$$

Here W_i is the remaining work of requests outstanding at i, which the router tracks itself. E_i, P_i, and TPOT_i are the Phase 1 profiles for vision encoding (v_r image tokens), prefill (p_r prompt tokens), and decode at the current batch size b_i. $\hat{o}_r$ is the predicted output length, taken as the per-class median on dev; an oracle variant bounds the loss from misprediction. s_r is the payload size, and C_i is the re-prefill penalty when r's session state is not resident on i. The router sends r to $\arg\min_i \mathrm{ECT}_i$, and a per-endpoint EWMA corrects predicted against observed time. A feasibility filter removes endpoints whose model does not fit or, in the extension, whose model tier misses the class's quality target.

Baselines. Always-i for each endpoint, round-robin, least-outstanding-requests, and weighted round-robin tuned on dev.

Evaluation. Steady and bursty arrivals on held-out mixes, with the fastest endpoint both idle and busy, using the Section 4 metrics. Ablations remove the queue, network, and session terms and swap predicted for oracle lengths. The final set of terms follows the Phase 1 findings; if a simple policy suffices, we report that result.

Risks. (1) Qwen3.5's vision path in llama.cpp is not yet verified on every backend; unsupported cases are reported as findings and served text-only. (2) Laptop and WAN variability (thermals, Wi-Fi, background load) is handled with fixed network setups, repeated runs, and reported variance. (3) Budget: the team's $5 \times \$25 = \$125$ of AWS credit buys about 150 cluster-hours of A+B+C+R at about \$0.83/h on-demand (to be confirmed): roughly 25 h for Phase 1, 90 h for Phase 2, and 35 h in reserve. We develop on laptops, auto-shut down instances, and set billing alarms.

References

[1] llama.cpp. `https://github.com/ggml-org/llama.cpp`

[2] Qwen Team. Qwen3.5 model collection. `https://huggingface.co/collections/Qwen/qwen35`

[3] M. Harchol-Balter. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge Univ. Press, 2013.

Appendix A: Timeline and Milestones

Due dates: Phase 1 Nov. 4, Phase 2 Dec. 10. Stages overlap; Thanksgiving (Nov. 26–29) is buffer.

Stage	Work and milestones
1. Environment setup Oct 27–Nov 1	Confirm instance eligibility and vCPU quota; scripted provisioning and teardown; pinned llama.cpp builds (Arm, x86, Metal, CUDA); checksummed GGUF and `mmproj` files; tunnel to off-cloud endpoints; load generator, JSONL logging, utilization sampler; pass-through round-robin router. **M1 (Nov 1):** every endpoint serves 0.8B text and image requests through the router.
2. Config. testing & expectation setup Nov 1–15	E1–E4 sweeps over models, quantizations, lengths, image resolutions, load, network, and session state (cold vs. warm); fit service-time models; freeze dev/held-out traces; mark H1–H4 supported or refuted. **M2 (Nov 4):** Phase 1 report. **M3 (Nov 15):** full profiles and frozen traces.
3. Router development Nov 12–25	Baselines; ECT router with queue, network, and session terms; output-length predictor; quality-constrained tier selection (extension); router-overhead microbenchmarks. **M4 (Nov 25):** router and all baselines run end-to-end on dev mixes.
4. Experiments & analysis Nov 23–Dec 10	Held-out runs (steady and bursty; fastest endpoint idle and busy); ablations; repeated runs with CIs; failure analysis; figures; report and artifacts. **M5 (Dec 5):** all runs complete. **M6 (Dec 10):** final report.